Week 4 Assignment — Answers

Section 1: Aptitude

1. Train crossing a platform

Train length = 150 m

Speed = 150 /15 =10 m/s

Platform length = 250 m

Total distance = 150+250 = 400 m

Time = 40 seconds

2. A and B complete a job together

A completes the work in 12 days.

B completes the work in 18 days.

Combined work per day = 1/12 + 1/18 = 5/36

Total time = 36/5 = 7.2 days.

2. Find the next number

Sequence : 2,6,12,20,30, ?

Differences : 4,6,8,10,12

Next number = 30+12 =42

4. Selling price at 15% profit

Cost price = 800

profit = 15% of 800 = 0.15 × 800 = 120.

Selling price = 800 + 120 = 920.

5. Coding

CODE = DPEF

Each letter moves forward by one position in the alphabet.

BUGS = CVHT

6. A sum doubles in 5 years at simple interest

Amount after 5 years = 2P

Simple interest earned = 2P – P = P

Time required to earn interest of 3P =15 years

7. Marked 40% above cost, then 20% discount

cost price = 100.
Marked price = 100 + 40% = 140.
Selling price after 20% discount = 140 × 0.80 = 112.
Profit = 112 − 100 = 12

8. Two trains crossing in opposite

directions

Combined length = 120 + 180 = 300 m.

Relative speed = 40 + 50 = 90 km/h
Speed in m/s = 25 m/s
Time = 300 ÷ 25 = **12 seconds**.

9. Trailing zeros in 100!

Trailing zeros are counted by the number of factors of 5 in 100!.
100/5 + 100/25 + 100/125 = 20 + 4 + 0 = **24 trailing zeros**.

10. Remainder when 21^100 is divided by 7

Powers of 2 modulo 7 repeat in a cycle: 2, 4, 1 (cycle length 3).
100 ÷ 3 leaves remainder 1, so 21^100 has the same remainder as 2^1.
Remainder = **2**.

Section 2: Leet Code

The following Python solutions are included for all mandatory problems and the optional Hard problem.

1. Two Sum (Easy)

```
class Solution:
 def two Sum(self,  nums : list[int], target: int) ->
list[int]:
      seen = {}
```

```
 for i, num in
enumerate(nums) :
 (nums ):
        needed = target _if
needed in seen:
            return [seen[needed], i]
seen[num] = i
```

Idea: Store each number's index in a hash map. For each number, check whether its complement has already appeared. Time: O(n); space: O(n).

20. Valid Parentheses (Easy)

```
class Solution:    def
isValid(self, s: str) -> bool:
      stack = []       pairs = {')': '(', ']':
'[', '}': '{'}

      for ch in s:
if ch in '([{':
            stack.append(ch)          else:            if
not stack or stack.pop() != pairs.get(ch):
              return False
return not stack
```

Idea: Push opening brackets onto a stack. Every closing bracket must match the most recent opening bracket. Time: O(n); space: O(n).

3. Longest Substring Without Repeating Characters (Medium)

```
class Solution:    def
lengthOfLongestSubstring(self, s: str) -> int:
      last_seen = {}
left = 0       best
= 0

      for right, ch in enumerate(s):          if ch
in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
last_seen[ch] = right          best =
max(best, right - left + 1)

      return best
```

Idea: Use a sliding window and remember each character's latest index. Move the left edge past a repeated character. Time: O(n); space: O(k), where k is the character set/window size.

Section 3: System Design — URL Shortener

Goal: Build a service that converts a long URL into a short link and redirects visitors to the original URL.

1. High-level flow

Create: Client sends a long URL → API validates it → service generates a short code → code and URL are saved in the database → API returns the short URL.
Redirect: Visitor opens the short URL → service looks up the code (cache first, then database) → responds with an HTTP redirect to the original URL.

2. API design

Create short link

POST /api/urls Request JSON: {"url": "https://example.com/some/long/page"}
Success response (201): {"short_url": "https://sho.rt/Ab3xYz"}
Validate that the URL is well-formed and allow only supported schemes such as HTTP and HTTPS.

Redirect

GET /{code}
Look up the code and return 301 (permanent) or 302 (temporary) redirect with a Location header. A 302 redirect is a flexible default when destinations may change or click analytics are needed.

3. Short-code generation and collision handling

Generate a random 7-character code from a URL-safe alphabet (for example, letters and digits). Store the code under a UNIQUE constraint. If an insert fails because the code already exists, generate a new code and retry a limited number of times. Another option at larger scale is to allocate a unique numeric ID and encode it using Base62. Do not rely on checking uniqueness in application code alone; enforce it in the database.

4. Database schema

urls ---id BIGINT PRIMARY
KEY short_code VARCHAR(16)
UNIQUE NOT NULL long_url

```
  TEXT NOT NULL created_at
  TIMESTAMP NOT NULL
  expires_at    TIMESTAMP NULL
  user_id       BIGINT NULL
  click_count   BIGINT NOT NULL
  DEFAULT 0
```

The short_code field is indexed (and unique), making lookups fast. For production, click events can be stored separately in a click_events table or sent to an analytics pipeline instead of updating click_count on every redirect.

5. Caching popular links

Use Redis as a cache with short_code → long_url entries. On redirect, check Redis first; on a cache miss, query the database and populate the cache. Apply a TTL (time-to-live), invalidate or update the cache if a destination changes, and use a negative-cache entry briefly for unknown codes to reduce repeated database lookups. Popular links stay in memory, lowering database load and redirect latency.

6. Reliability, security, and scaling

- Add rate limits to prevent abuse and automated link creation.
- Validate URLs and consider blocking malicious destinations; protect against open-redirect abuse.
- Use HTTPS, monitoring, logs, backups, and database replication as traffic grows.
- Keep redirect handling lightweight; process detailed analytics asynchronously.
- If the service is distributed, use a shared cache and a database with a unique constraint on short_code.